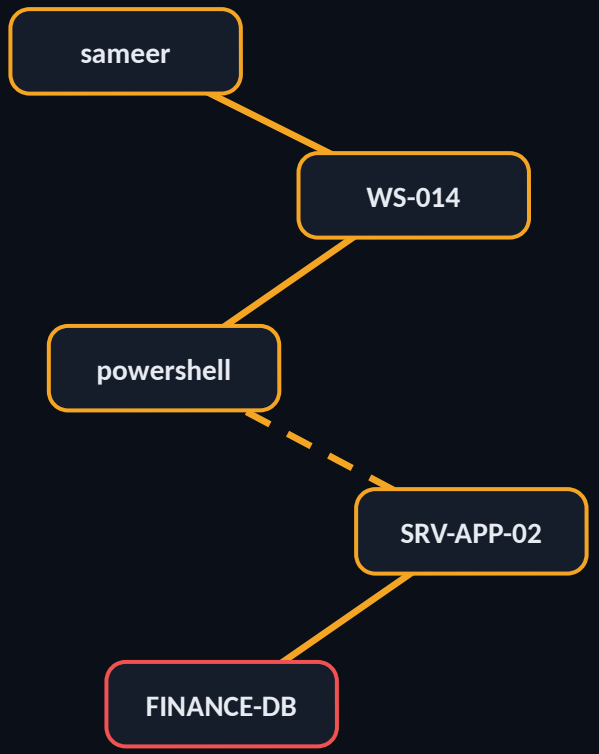
AttackGraph Lens

From fragmented security events to explainable attack paths and blast-radius intelligence.

hackFront India 2026 | T-2 Cybersecurity | T2.1 GraphOps

Who was reached, in what order, and how do we know?



`20:40 → 20:45 → 21:22 → 21:32 UTC`



Alerts arrive in pieces; the attack happens as a chain

Seven feeds, seven partial views

Identity
who logged on where

Endpoint / EDR
what ran, what it touched

Firewall
who talked to whom

DNS
which names resolved

Database audit
who read which data

SIEM alerts
what looked bad

Asset inventory
what matters, IP ↔ host

T2.1 GraphOps asks for three things

1. **Attack-path discovery**
   How did the attacker get from the first foothold to the crown jewels?
2. **Real-time blast radius**
   If this system is compromised, what else is now at risk?
3. **Unified incident intelligence**
   Turn scattered alerts into one story an analyst can act on.

WHY Each tool sees one slice. The attack is the chain across them, and today an analyst rebuilds that chain by hand.

Connectivity is not compromise: time and evidence decide

Rule 1 · Temporal feasibility

A step must occur after the event that enables it.



Graph says: route exists. **Time says: impossible, rejected ✕**

Rule 2 · Evidence provenance

A relationship without source records is not proof.

`edge`	powershell.exe#7344 → CONNECTED_TO → SRV-APP-02
`rule`	R-NET-01 (EDR network connect) + R-RES-01 (IP → host)
`record`	edr:EDR-000399 · edr_events.jsonl, row 270
`basis`	INFERRED: 10.10.2.12 resolved via asset inventory
`why bad`	cited by alert ALR-0006 (high), suspicion 0.75

WHY Most graph views stop at “A can reach B”. Analysts need “A did reach B, after X, and here is the record”.

Where AttackGraph Lens goes beyond conventional graph-based security analysis

A

Temporal feasibility of attack paths

- Keeps every occurrence and timestamp
- Enforces order and a maximum gap
- Rejects impossible routes, with the reason

B

Provenance-native evidence graph

- Edges keep source-record references
- Observed vs inferred kept separate
- Conclusion → file, row, rule, event

C

Evidence-tiered blast radius

- Confirmed · Suspected · Observed · Potential
- Admin reach ≠ compromise
- Confirmed only by an analyst

D

Multichannel corroboration

- 6 channels: identity to detection
- Primary, corroborating, silent, blind
- Bounded MD deviation as support

E

Counterfactual, time-aware containment

- Simulate an action at time T
- Re-run paths and blast radius
- Risk reduction vs disruption cost

Not merely a graph of possible connections: an evidence-backed reconstruction of what could have happened, what likely happened, and why.

NOVELTY AttackGraph Lens integrates temporal feasibility, occurrence-level evidence provenance, graded blast-radius reasoning, multichannel corroboration and time-aware counterfactual containment into one explainable security investigation workflow.

The building blocks (temporal graphs, graph traversal, Mahalanobis distance, alert correlation, counterfactual analysis) are established techniques. The contribution is their integration.

Built on previous cybersecurity work, designed for explainable investigation

UniAethel

Previous cybersecurity project

- A cybersecurity tool previously developed by the team
- Shortlisted at SUTRAM 2026 by NFSU
- Separate project, separate code, different objectives

Design philosophy

What carried over

- Evidence-oriented security thinking
- Analyst-centric interaction
- Clear technical presentation

AttackGraph Lens

What is new for T2.1

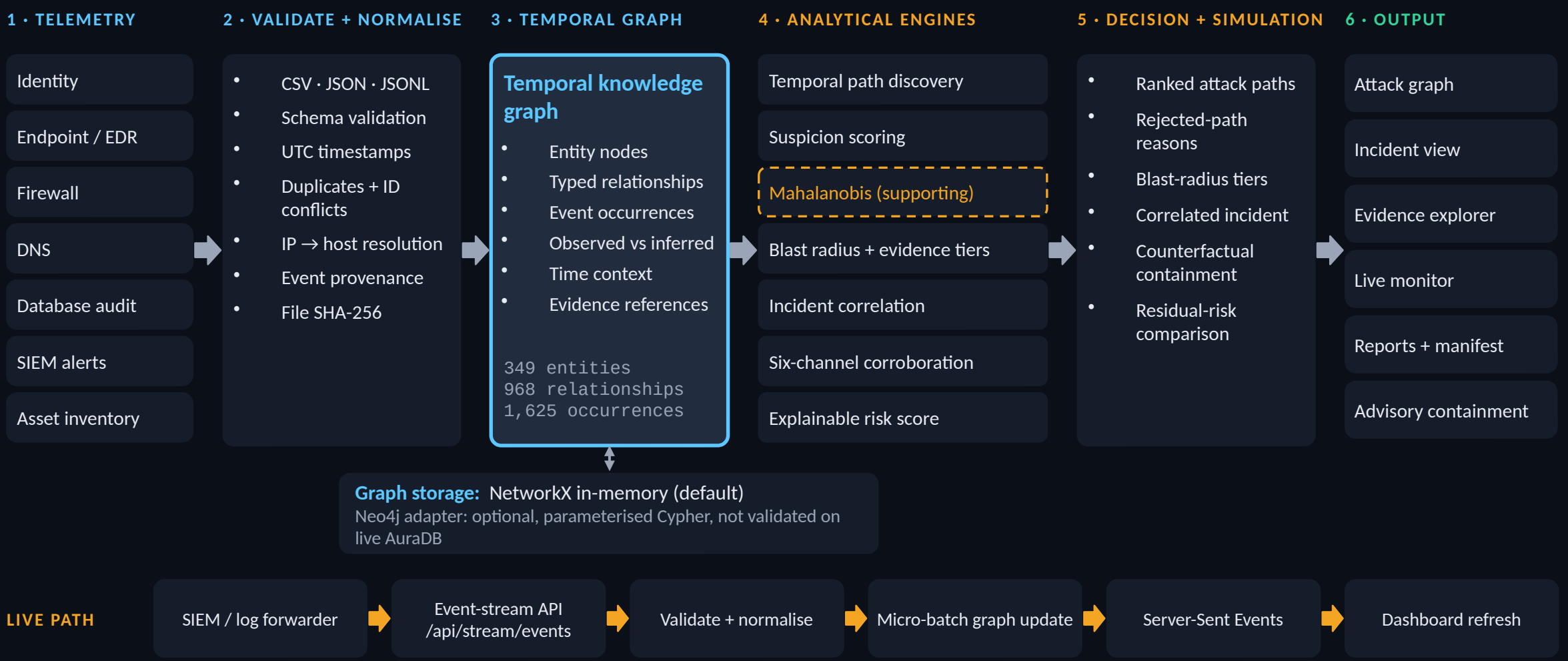
- Focused GraphOps investigation engine
- New code, synthetic dataset and methods
- No UniAethel code, data or results reused

“Design philosophy inspired by UniAethel, a previous cybersecurity tool shortlisted at SUTRAM 2026 by NFSU.”

NOTE

The shortlist recognises UniAethel only. It does not validate AttackGraph Lens, and NFSU has not developed, endorsed or reviewed this project.

Six layers around one temporal knowledge graph



Input → output in six stages, each with one job

1. **Ingest** — hash, validate, reject
2. **Normalise** — UTC, one schema, IP → host
3. **Graph** — entities, edges, occurrences
4. **Suspicion** — s = max(4 signals) ≥ 0.5
5. **Reason** — paths, blast, correlate
6. **Explain** — score, evidence, report

1,258 rows read → 1,251 accepted (5 rejected, 1 duplicate, 1 conflict) → **1,191 events** + 60 inventory assets → **349 entities · 968 relationships · 1,625 occurrences**

Source · file	Fields read	Becomes in the graph
Identity · auth_logs.csv	`user, src_host, dst_host, logon_type, result`	`user → AUTHENTICATED_TO → host`
Endpoint · edr_events.jsonl	`host, image, pid, parent, cmdline, dst ip:port`	`host → EXECUTED → process → SPAWNED / CONNECTED_TO`
Firewall · firewall.csv	`src_ip, dst_ip, dst_port, action, bytes_out`	`host → CONNECTED_TO / COMMUNICATED_WITH → host | ip`
DNS · dns.json	`client_ip, query, answers`	`host → QUERIED → domain → RESOLVED_TO → ip`
Database · db_audit.csv	`db_user, client_host, database, rows_returned`	`host → ACCESSED → database (volume = rows)`
SIEM · siem_alerts.json	`rule, severity, MITRE, entities, evidence_refs`	`alert → ALERTED_ON → entity; severity to cited events`
Inventory · asset_inventory.csv	`name, ip, asset_type, criticality 1 to 5`	`IP → host resolution; criticality for risk`

WHY Rows, events and occurrences are different counts: inventory rows are reference data, and one event can create several relationships.

One raw record becomes one explainable graph edge

1 · RAW RECORD (edr_events.jsonl, row 270)


```
{"id": "EDR-000399",
 "ts": "2026-10-27T21:22:00Z",
 "host": "WS-014",
 "user": "sameer",
 "event_type": "network_connect",
 "image": "powershell.exe",
 "pid": "7344",
 "dst_ip": "10.10.2.12",
 "dst_port": 5985,
 "severity": "medium"}
```


2 · NORMALISE + RESOLVE

time	21:22:00 UTC
process	WS-014/ powershell.exe#7344
dest IP	10.10.2.12
host	SRV-APP-02 (inferred, inventory · R-RES-01)
severity	"medium" → 0.50
source	file SHA-256 + row 270

3 · EVIDENCE-BACKED EDGE



CONNECTED_TO (dashed = inferred)

rule	R-NET-01
basis	INFERRED
event	edr:EDR-000399
native	0.50 (medium)
alert	ALR-0006 cites it: 0.75
suspicion	**0.75 ≥ 0.5** ✓

WHY Click any edge in the app and you land on this exact record: the conclusion and its proof are one click apart.

Suspicious = the strongest of four signals reaches 0.5

$$s(o) = \max(s_{native} , s_{alert} , 0.8 \cdot s_{proximity} , s_{MD})$$

$$\text{suspicious} \Leftrightarrow s(o) \geq \tau = 0.5$$

R-SUSP-00

Native severity

The source's own label: info .05 · low .25 · medium .50 · high .75 · critical .95

R-SUSP-01

Alert reference

An alert lists this record as evidence, so it inherits the alert severity (unless dismissed as benign)

R-SUSP-02

Alert proximity

Both endpoints named by one alert, inside its time window: 0.8 × alert severity

R-SUSP-03

Mahalanobis deviation

Behaviour far from its own baseline: at most 0.45, so it can support but never decide

EDR-000399 (WinRM to SRV-APP-02)

$\max(0.50, 0.75, 0, 0) = 0.75 \rightarrow$ suspicious

svc_backup → BKP-01 (nightly backup)

$D^2 = 66 \rightarrow s_{MD} = 0.45 < 0.5 \rightarrow$ not suspicious

WHY Maximum, not average: one strong independent signal is enough. Averaging would dilute a real alert with quiet noise.

How unusual is this behaviour against its own history?

features $x = [\sin(2\pi h/24), \cos(2\pi h/24), \log_{10}(1+vol), novelty]$

hour wraps round the clock; volume on a log scale

baseline same pair if $n \geq 20$, else same type if $n \geq 30$

an entity is compared with its own history first

robust z $z = (x - median) / (1.4826 \cdot MAD)$

outliers cannot poison the baseline

shrinkage $S = 0.8 \cdot cov(z) + 0.2 \cdot I$

stable inverse even with few samples

distance $D^2 = z^T S^{-1} z$

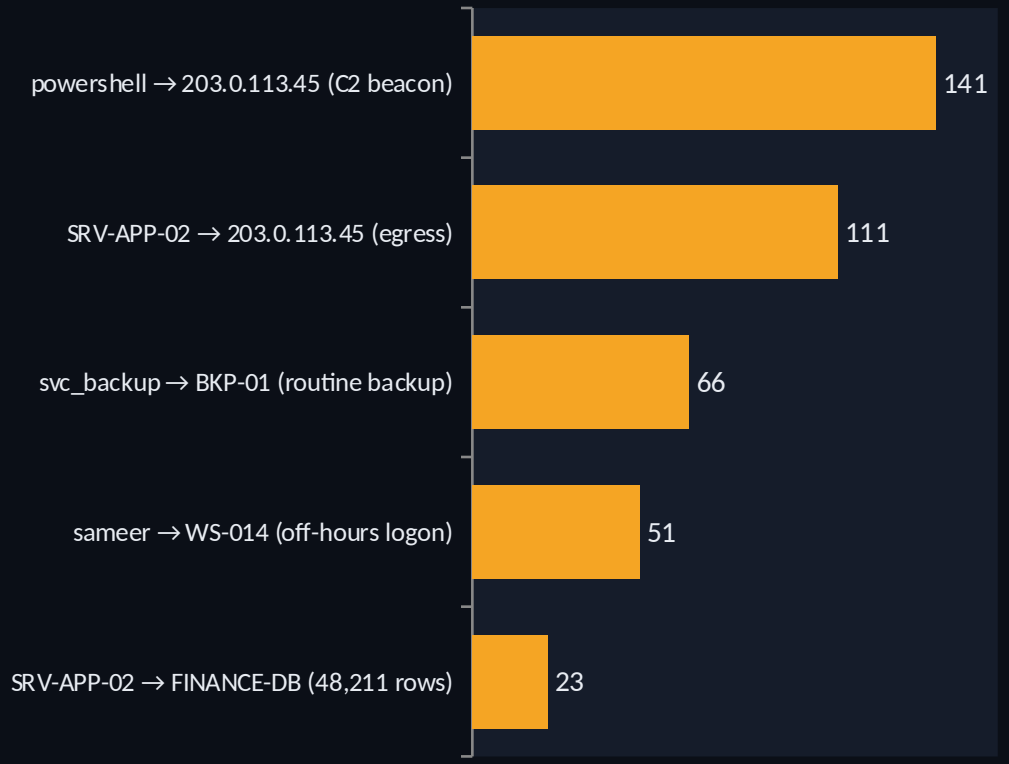
joint unusualness, correlations accounted for

score $s_{MD} = 0.45 \cdot \min(1, (D^2 - 18.47) / 18.47)$

bounded: supports a decision, never makes it

18.47 = χ^2 (4 d.f.) 99.9th percentile. The distance is standard; the bounding is ours.

Mahalanobis D^2 on the synthetic dataset (reference 18.47)



WHY Deviation shows what is unusual for an entity's own history; the bounded contribution stops anomaly detection from convicting an event on its own.

A path is valid only if every step can follow the previous one

$$t(i) - 120\ s \leq t(i+1) \leq t(i) + 6\ h$$ clock skew allowed · stale links cut after 6 h

Valid hypothesis #2 · sameer → FINANCE-DB

20:40 — `sameer → AUTHENTICATED_TO → WS-014`
off-hours VPN logon

20:45 — `WS-014 → EXECUTED → powershell #7344`
encoded command

21:22 — `powershell → CONNECTED_TO → SRV-APP-02`
WinRM, port 5985

21:32 — `SRV-APP-02 → ACCESSED → FINANCE-DB`
48,211 rows as svc_backup

Rejected · via BKP-01

BKP-01 → FINANCE-DB last occurred 19:30, but the attacker could reach BKP-01 at 21:50 at the earliest. Shown to the analyst with this reason.

8 valid paths	**3** rejected
138 expansions	**2.3 ms** query

measured on the synthetic dataset, local machine

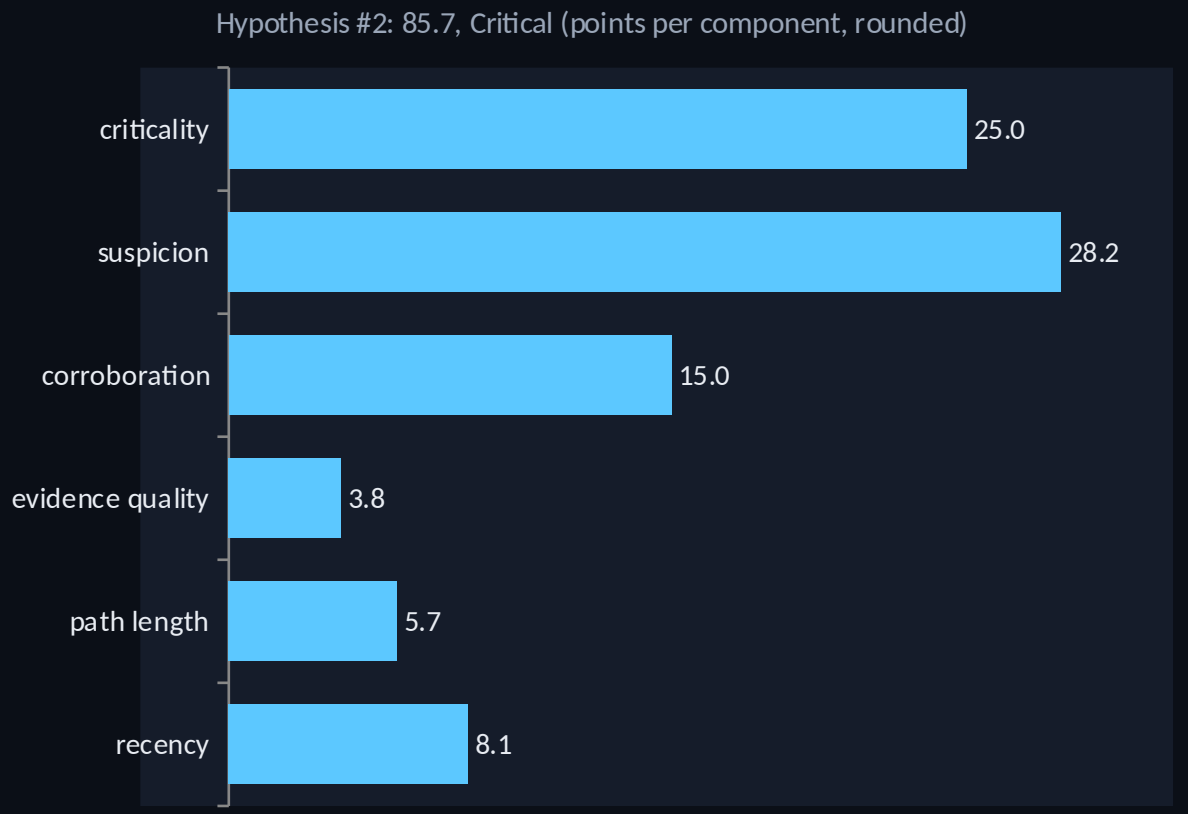
WHY A static graph answers whether a route exists. AttackGraph Lens checks whether the sequence is temporally feasible.

Every point of the score is visible and explained

$$\text{Score} = 100 \cdot \Sigma\, W_k \cdot C_k \quad \text{each } c_k \text{ in } [0, 1]$$

.25	**criticality**	most critical asset reached (1 to 5)
.35	**suspicion**	share and strength of suspicious hops
.15	**corroboration**	independent source systems
.05	**evidence quality**	observed vs inferred hops
.10	**path length**	shorter = more direct
.10	**recency**	half-life 48 h

Critical ≥ 75 | High ≥ 55 | Moderate ≥ 35 | Low



WHY This is an explainable prioritisation score, not a calibrated probability of compromise. Every weight is visible and adjustable.

Same reach, different meaning: four evidence tiers

CONFIRMED_COMPROMISE

Only when an analyst marks it. Never automatic.

SUSPECTED_LATERAL_MOVEMENT

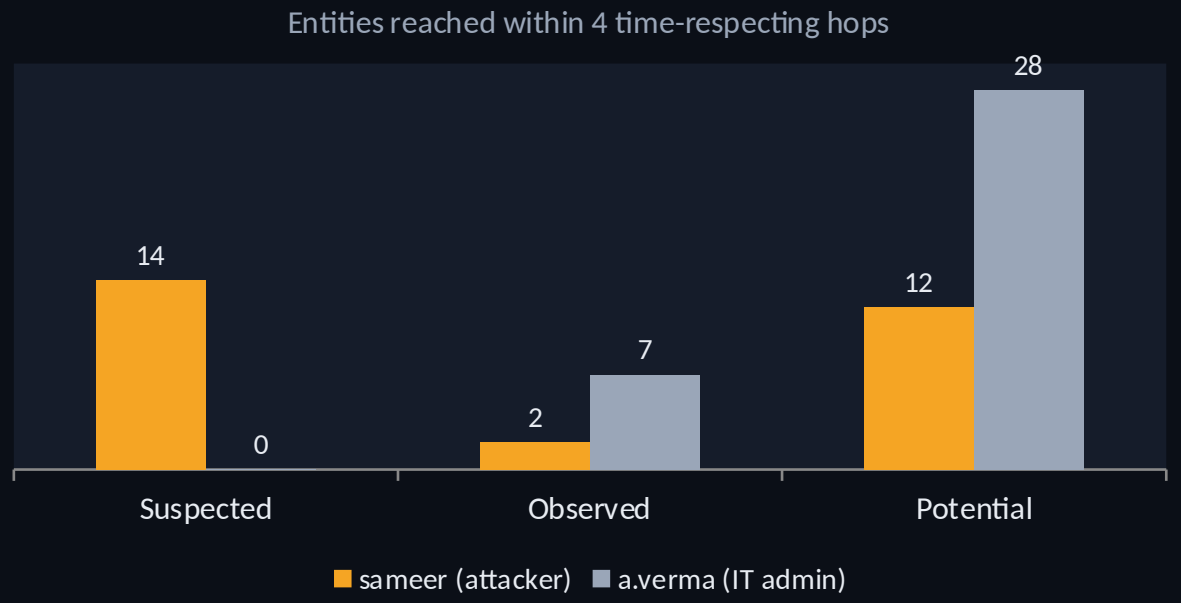
Reached through an unbroken chain of suspicious steps.

OBSERVED_CONNECTION

Direct contact (hop 1), nothing suspicious on it.

POTENTIAL_EXPOSURE

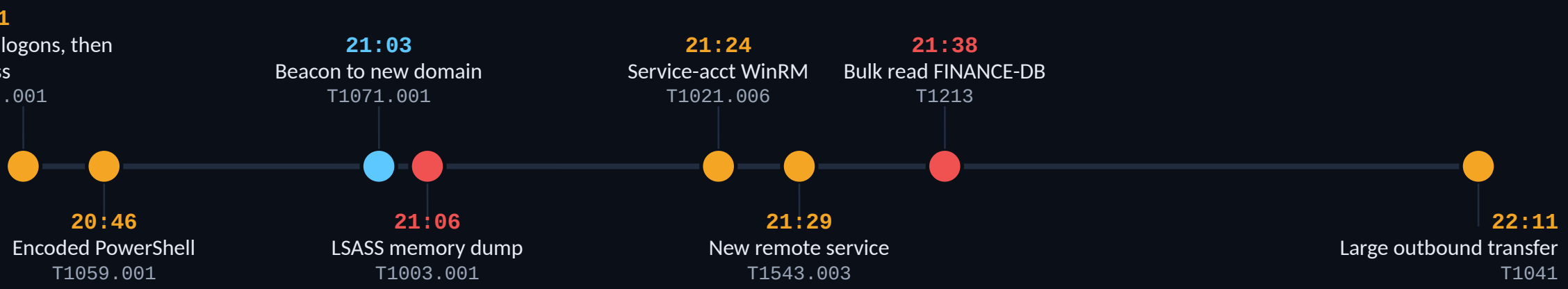
Reachable by ordinary activity: exposure, not evidence.



FINANCE-DB: SUSPECTED from sameer (hop 3) vs POTENTIAL from the admin (hop 2).

WHY Simple transitive reachability would flag every asset an admin can touch. Tiers separate “could” from “did, with evidence”.

8 alerts from 6 sources become one incident: INC-0002



R-COR-01

Shared entity

two signals touch the same non-hub entity

R-COR-02

Graph adjacency

neighbouring entities within 90 min

R-COR-03

Explicit reference

an alert cites the event as evidence

Hub exclusion

DC-01, FS-01

degree ≥ max(20, 4 × median) cannot glue incidents

WHY Union-find over cited rules gives INC-0002: Critical 92.6 · 2 identities · 5 assets · 6 independent sources. One story, not 8 tickets.

Who else saw it? Six independent channels per step

Step	Identity	Endpoint	Network	DNS	Data	Detection
1 · logon 20:40	■ primary	● corrob.	● corrob.	● corrob.	none	● corrob.
2 · exec 20:45	● corrob.	■ primary	● corrob.	● corrob.	none	● corrob.
3 · WinRM 21:22	● corrob.	■ primary	● corrob.	none	● corrob.	● corrob.
4 · DB read 21:32	● corrob.	● corrob.	● corrob.	none	■ primary	● corrob.

100%

channel agreement

4 / 4

steps seen by ≥ 2 channels

0

expected-but-silent gaps

■ primary

the channel that created the edge

● corroborating

independent record, same entities, within ±15 min

? silent

expected to see it, logged nothing: an investigative lead, not proof

blind

cannot observe this entity type at all

WHY One tool can be wrong or blinded. Independent channels are hard to fake, and a silent channel is worth a look.

What to cut depends on when you act

Act at 20:41 · first alert

1 action

Attacker holds: sameer, WS-014

Isolate host WS-014

`−98.7 % path risk · paths 57 → 1 · critical 48 → 0`

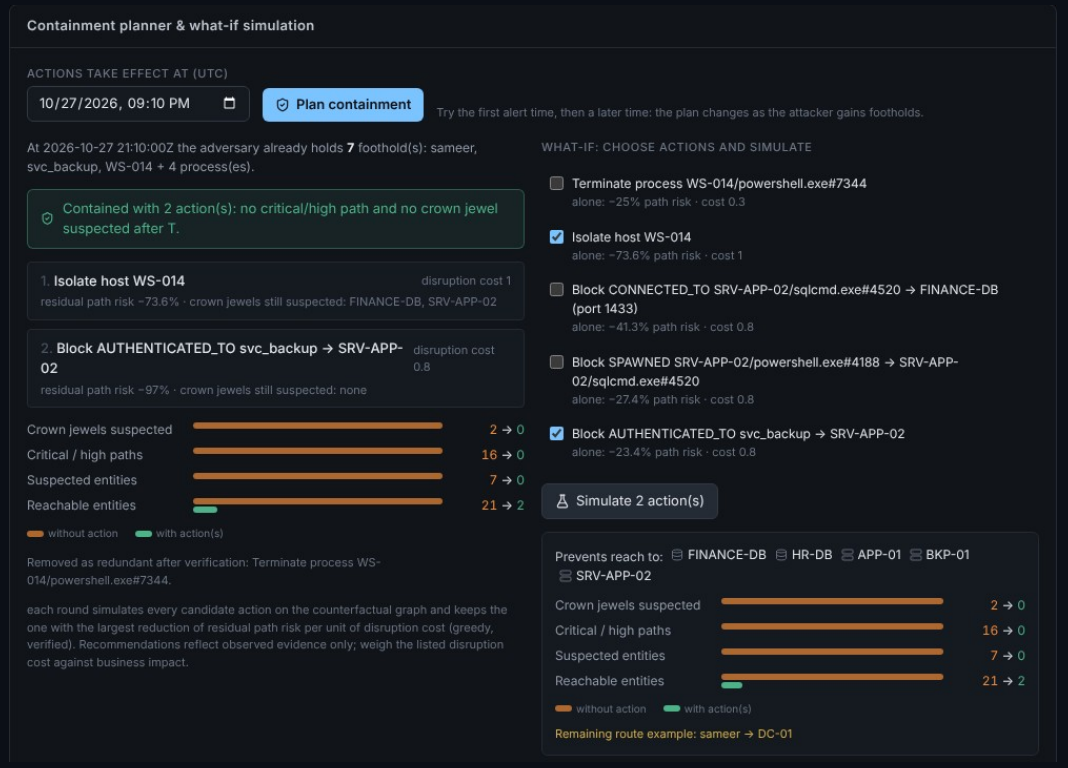
Act at 21:10 · 30 min later

2 actions

LSASS dumped at 21:06: svc_backup credential stolen · 7 footholds

Isolate WS-014 + block svc_backup → SRV-APP-02

`critical/high paths 16 → 0 · crown jewels 2 → 0`

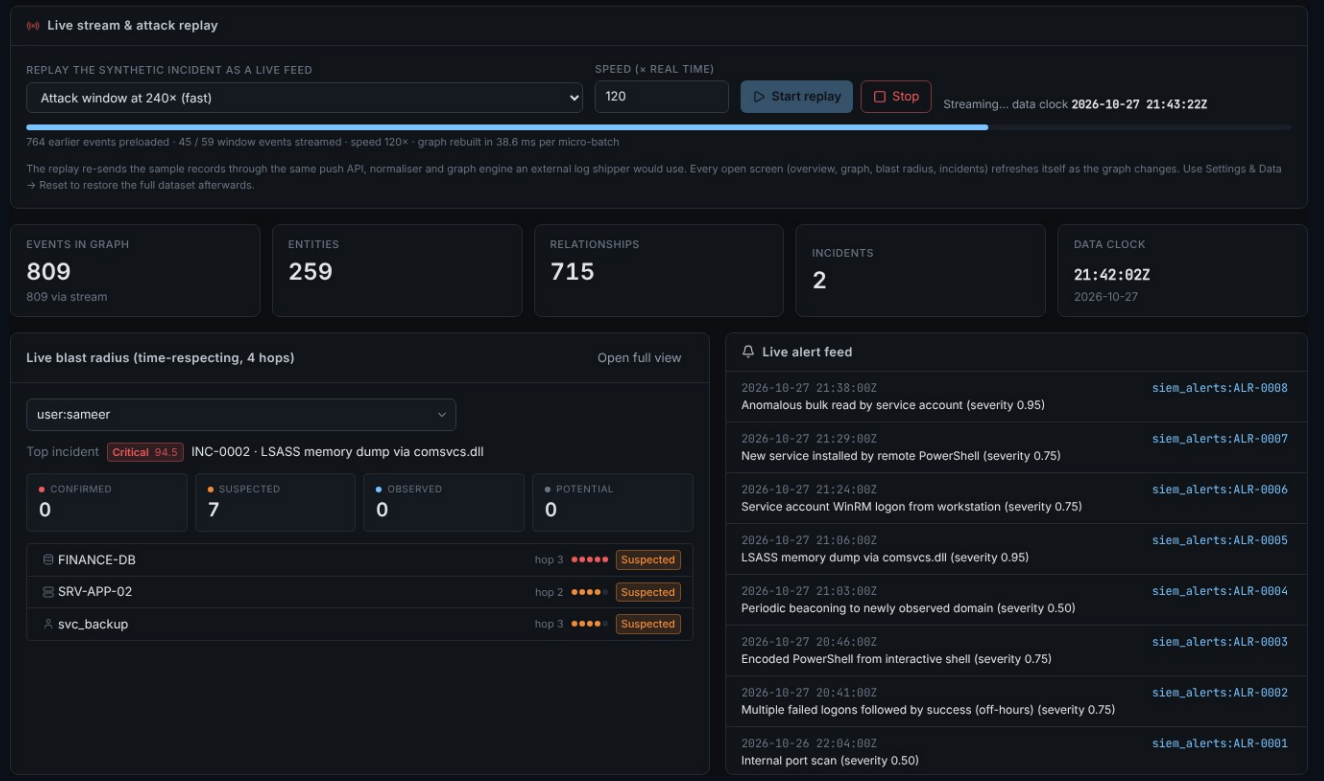


Method: remove every occurrence after T that touches the contained entity, re-run paths and blast radius, then pick the best risk reduction per unit of disruption cost.

WHY Advisory only: the tool simulates and recommends; it never isolates a system. It shows the smallest cut that stops the modelled attack.

The picture builds while the attack is still happening

1. **`POST /api/stream/events`**
   SIEM forwarder or log shipper pushes records
2. **Same validation + normaliser**
   duplicates and bad rows rejected
3. **Micro-batch every 0.4 s**
   graph rebuilt; 38.6 ms per batch in replay*
4. **Version + 1 → Server-Sent Events**
   browser notified, refresh at most 1/s
5. **Every view updates**
   blast radius, incidents, paths, graph



*Replay of the synthetic intrusion on a local machine. When the 21:32 read streams in, FINANCE-DB turns SUSPECTED and INC-0002 goes Critical.

WHY T2.1 asks for the blast radius in real time. Live mode uses the same engine as batch analysis; there is no separate demo code path.

What we demonstrated, and what we do not claim

105
automated tests passing
(incl. 6 browser end-to-end)

95 %
backend statement
coverage

88 ms
ingest 7 files + build graph
(1,191 events)*

0.4 ms
4-hop temporal
blast radius*

Demonstrated (tests assert these)

- Principal path found, 4 of 4 hops suspicious, starts 20:40
- Backup route rejected as temporally impossible
- FINANCE-DB: SUSPECTED for attacker, POTENTIAL for admin
- Bad rows, duplicate and ID conflict flagged
- Exported report hash appears in the evidence manifest

Not claimed

- No accuracy, precision, recall or F1: the only labels are synthetic, so a score would be circular
- No enterprise-scale performance: timings are this dataset only
- No live Neo4j / AuraDB validation
- No analyst user study yet

WHY *Medians of 5 runs on the synthetic dataset, one local machine. A tool that overclaims is a liability; we state the limits plainly.

Built to be run and trusted, with its limits stated

Engineering

Engine

Python 3.11+ · FastAPI · NetworkX · NumPy

Interface

React · TypeScript · React Flow · dagre

Storage

in-memory; Neo4j optional (parameterised Cypher)

Controls

uploads validated, never executed; SHA-256 hashes

Delivery

run script · Docker · CI · MIT licence

Limits today

Data

synthetic dataset only

Backend

no live Neo4j / AuraDB validation

Scale

no enterprise-scale validation

Users

no analyst study; no authentication yet

Connectors

file + push API only; no native SIEM connectors

Next steps

Telemetry

real connectors: Sysmon, Zeek, cloud audit logs

Calibration

learn weights and τ from labelled data

Backend

validate the graph backend live

Volume

higher-volume streaming

People

analyst feedback loop; multi-user auth

NOTE Built with AI assistance (Anthropic Claude) under the team's direction; AI use and all third-party components are disclosed in the repository.

T2.1 asked three questions. Each has a traceable answer.

Map attack paths	Time-feasible paths with evidence-backed explanations; impossible routes rejected with the reason.
Assess blast radius	Four evidence tiers, updated as events arrive; attacker and admin told apart on the same asset.
Build one incident picture	Correlated alerts, independent corroboration and time-aware containment simulation.

Demo video: demo/attackgraph_lens_demo.mp4 · Methods: docs/METHODOLOGY.md · Validation: docs/VALIDATION_REPORT.md (in the submission ZIP and repository)

WHY Who was reached, in what order, and how do we know? AttackGraph Lens answers with the evidence behind every step.